Strings of Code from an external source, effectively building your own simple Ruby interpreter.

If you want to use Kernel#eval to its fullest potential, you should also learn about the Binding class.

Binding Objects

A Binding is a whole scope packaged as an object. The idea is that you can create a Binding to capture the local scope and carry it around. Later, you can execute code in that scope by using the Binding object in conjunction with eval.

You can create a Binding with the Kernel#binding method:

ctwc/bindings.rb

```
class MyClass
  def my_method
    @x = 1
    binding
  end
end

b = MyClass.new.my_method
```

You can think of Binding objects as "purer" forms of closures than blocks because these objects contain a scope but don't contain code. You can evaluate code in the captured scope by passing the Binding as an additional argument to eval:

```
eval "@x", b        # => 1
```

Ruby also provides a predefined constant named TOPLEVEL_BINDING, which is just a Binding of the top-level scope. You can use it to access the top-level scope from anywhere in your program:

```
class AnotherClass
  def my_method
    eval "self", TOPLEVEL_BINDING
  end
end

AnotherClass.new.my_method    # => main
```

One gem that makes good use of bindings is Pry, which you met in *The Pry Example*, on page 49. Pry defines an Object#pry method that opens an interactive session inside the object's scope, similar to what irb does with nested sessions. You can use this function as a debugger of sorts: instead of setting a breakpoint, you add a line to your code that calls pry on the current bindings, as shown in the following code.